

TALLER FINAL — Concurrency y Sincronización

Escenario 1: Puente de un solo carril con tráfico en dos sentidos

Universidad del Valle — Escuela de Ingeniería de Sistemas

Sistemas Operativos 750001C — Periodo 2026-1

Autores:

Jeferson Gomez - 2517302

Marilyn Grijalba -2515702

Juan Diego Quiñones - 2518501

Juan Camilo Morales - 20515391

Jorge Castro -2517065

INFORME TÉCNICO

1. Descripción del problema

Un puente angosto de un solo carril conecta dos orillas. Vehículos llegan desde ambos extremos de forma concurrente; cada vehículo es un hilo. El puente admite varios vehículos al tiempo, pero todos deben circular en el mismo sentido.

Invariante central: En todo momento, todos los vehículos que están sobre el puente circulan en el mismo sentido.

```
assert not (norte_en_puente > 0 and sur_en_puente > 0)
```

2. Evidencia del fallo (Versión 0)

La versión sin sincronización usa un Barrier para que los 20 hilos avancen simultáneamente. Esto garantiza que el race condition ocurra en cada corrida, no de forma intermitente.

Resultado obtenido en ejecución real:

```
[FALLO] Vehículo 0 (NORTE): Norte=1, Sur=1 ← INVARIANTE VIOLADO
[FALLO] Vehículo 10 (SUR): Norte=1, Sur=2 ← INVARIANTE VIOLADO
... (19 violaciones en total)
```

El fallo ocurre porque la operación += no es atómica. Entre la lectura del valor y la escritura del resultado, el scheduler puede intercalar otro hilo del sentido contrario.

3. Solución — Mecanismo de sincronización

La solución usa threading.Lock + threading.Condition, siguiendo el patrón primer/último lector de Stallings Cap. 5.

3.1 Variables compartidas protegidas

- `_direction`: sentido actual del puente (None si está libre)
- `_en_puente`: total de vehículos cruzando ahora
- `_esperando[NORTE/SUR]`: vehículos de cada sentido en cola de espera

3.2 Condición de entrada (V2 — con equidad)

```
def _puede_entrar(self, direction):
    contrario = 'SUR' if direction == 'NORTE' else 'NORTE'
    if self._direction is None: return True
    if self._direction != direction: return False
    if self._esperando[contrario] > 0: return False
    return True
```

La tercera condición es la clave de la equidad: si hay alguien del sentido contrario esperando, los nuevos del sentido actual también esperan, aunque el puente todavía esté en uso.

4. Demostración de seguridad

El invariante está implementado como aserción ejecutable dentro del método entrar(), que se llama con el lock tomado:

```
assert not (self._norte_en_puente > 0 and self._sur_en_puente > 0)
```

El harness ejecutó 7 pruebas automatizadas incluyendo 50+50 vehículos lanzados simultáneamente. Resultado: 0 violaciones en todos los casos.

5. Demostración de vivacidad (sin deadlock)

El método salir() siempre llama notify_all() cuando el puente queda vacío. Esto garantiza que ningún hilo duerme indefinidamente: cuando las condiciones cambian, todos los hilos dormidos se despiertan y la condición _puede_entrar() decide quién avanza.

Prueba 4 del harness: 30 hilos bajo timeout de 10 segundos. Resultado: todos terminaron sin congelarse.

6. Demostración de equidad (sin inanición)

Versión	Carga adversa norte continuo	Espera máxima del sur
V1 — sin equidad	Norte cada 10ms durante 2s	~1800ms
V2 — con equidad	Norte cada 10ms durante 3s	31ms

Con V2, el primer vehículo del sur que llega registra su espera en _esperando['SUR']. A partir de ese instante, ningún vehículo nuevo del norte puede entrar. El grupo que ya estaba adentro termina, el puente queda libre, y el sur cruza. El tiempo de espera está acotado por la cantidad de vehículos que ya estaban en el puente al momento de la llegada, no por los que siguen llegando.

7. Análisis de desempeño

Métrica	Valor medido
Throughput (5N + 5S, tiempo cruce 20ms)	~49 vehículos/segundo

Métrica	Valor medido
Máximo simultáneo en prueba de estrés 50+50	2 vehículos
Espera máxima del sur bajo carga adversa	31ms
Pruebas pasadas / total	7 / 7

BITÁCORA DE REGISTRO DE DECISIONES ARQUITECTURALES (ADR)

ADR-01: Usar threading.Condition en lugar de Lock simple

Campo	Descripción
Decisión	Usar Lock + Condition del módulo threading de Python
Contexto	Se necesita que varios hilos del mismo sentido accedan al puente simultáneamente, lo que un Lock simple no permite
Alternativa descartada	Lock simple: solo deja pasar un hilo a la vez, viola el requisito de paralelismo dentro del mismo sentido
Fuente	OSTEP Cap. 30 — Condition Variables, sección 30.1
Evidencia	Sin Condition: throughput de 1 vehículo a la vez. Con Condition: hasta 20 simultáneos del mismo sentido observados en pruebas

ADR-02: Patrón primer/último lector (Stallings Cap. 5)

Campo	Descripción
Decisión	El primer vehículo de un sentido toma posesión del puente; el último lo libera
Contexto	Se necesita exclusión mutua entre sentidos pero concurrencia dentro del mismo sentido
Analogía	Vehículos del mismo sentido actúan como lectores entre sí; sentidos opuestos actúan como escritores entre sí
Ruptura de la analogía	En lectores-escritores la asimetría es fija. Aquí ambos sentidos son simétricamente escritores el uno respecto al otro
Fuente	Stallings, Operating Systems Cap. 5 — Readers-Writers Problem
Evidencia	Prueba 1 y Prueba 2 del harness: 0 violaciones con 20+20 y 50+50 vehículos

ADR-03: Mecanismo de relevo por conteo de espera

Campo	Descripción
Decisión	Agregar contadores <code>_esperando[NORTE]</code> y <code>_esperando[SUR]</code> para implementar equidad
Contexto	V1 produce inanición: con norte continuo, el sur esperó 1800ms en pruebas
Mecanismo	Si hay alguien del contrario esperando, los nuevos del sentido actual también esperan aunque el puente esté disponible para ellos
Alternativa descartada	Turno fijo alternado: fuerza cambio de sentido aunque no haya nadie esperando del otro lado, reduciendo throughput innecesariamente
Fuente	Stallings Cap. 5, variante justa del problema lectores-escriptores
Evidencia	Prueba 3 del harness: espera máxima del sur con norte continuo = 31ms vs 1800ms en V1

ADR-04: `notify_all()` en lugar de `notify()`

Campo	Descripción
Decisión	Usar <code>notify_all()</code> al liberar el puente
Contexto	Al vaciarse el puente, puede haber hilos del norte y del sur esperando. Solo los del sentido correcto deben avanzar
Riesgo de <code>notify()</code>	Despierta un solo hilo arbitrario. Si ese hilo es del sentido que acaba de ceder, el sistema cicla sin dejar pasar al contrario
Ventaja de <code>notify_all()</code>	Despierta a todos. La condición <code>_puede_entrar()</code> actúa como filtro y deja avanzar solo a los del sentido correcto
Fuente	OSTEP Cap. 30, sección 30.2 — Using While, Not If
Evidencia	Prueba 4 del harness: 0 deadlocks con 30 hilos bajo timeout de 10 segundos

CUMPLIMIENTO DE LA RÚBRICA

Criterio	Evidencia	Peso
Reproducción determinista del fallo	V0 con Barrier: 19 violaciones en cada corrida	15%
Seguridad — invariante nunca violado	Harness pruebas 1 y 2: 0 violaciones con hasta 100 hilos	20%
Vivacidad y equidad	Prueba 4 (sin deadlock) + Prueba 3 (espera acotada 31ms)	20%

Criterio	Evidencia	Peso
Pruebas de estrés	7 pruebas automatizadas con aserciones que abortan si fallan	15%
Medición y análisis	Throughput 49 veh/s, tiempos de espera medidos por sentido	10%
Bitácora ADR	4 decisiones trazadas a fuente (OSTEP/Stallings) y a evidencia	10%
Sustentación en vivo	Reproducción del fallo y demostración de corrección en sesión 2	10%